SPRING 2024

PERFORMANCE PROFILING FOR C++ SERIALIZATION LIBRARIES

ROWAN NIGHTHAWKS

Prepared By :

Kareena Khan
Evan Marino
Aaron Boylan
Colin Harrell
Jacob Flores

Prepared For :

ASRC Federal
Daniel Dusharm

Table of Contents

1. Abstract 1
2. Test Bench Elements & Environment 2
3. Chosen Libraries & Previous Considerations 3
4. Observational and Researched Qualities Analysis 4
5. Metric Analysis Using the Test Bench 10
6. Overall Library Recommendation 15
7. Work Cited 16

Abstract

This report will analyze Open Source Serialization Libraries:

- Boost [1.82.0]
- Cereal [1.3.2]
- FlatBuffers [23.5.26]
- ProtoBuf [3.12]

Based upon Observed Qualities:

- Ease of Use
- Security Concerns
- Community Support

As well gathered performance metrics from a developed Test Bench:

- Speed
- Memory Usage
- Data Integrity

The Test Bench analyses are all done in a C++/Linux environment. From the various evaluations, recommendations will be made for specific use cases. As there are various techniques of Serialization such as Binary, and Textual, these recommendations can help highlight performance tradeoffs between the different serialization techniques, and introduce various C++ Serialization Libraries.

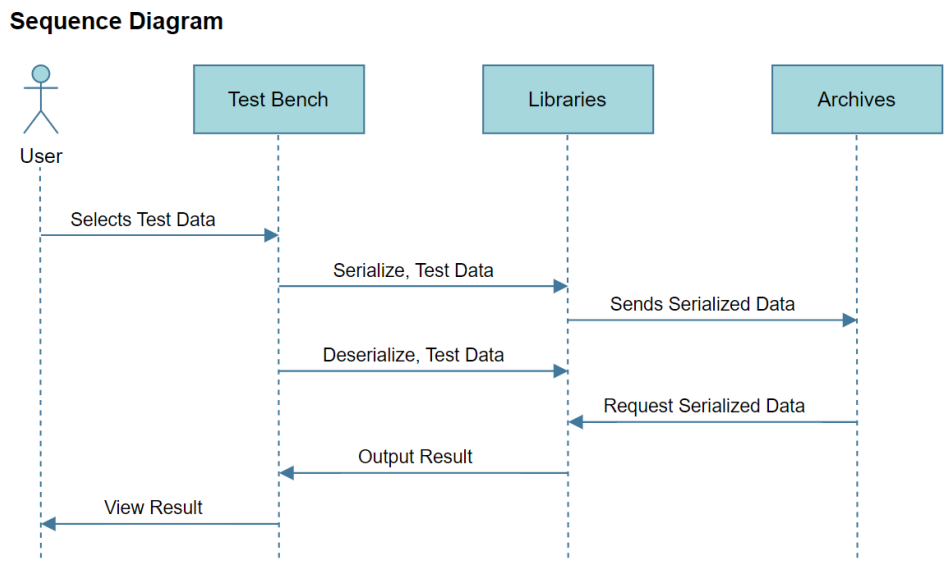
Test Bench Elements & Environment

The Test Bench is a program that our team developed that feeds data to the four aforementioned libraries for them to serialize and deserialize, and then display metrics about the performance of each library including speed, memory usage, and data integrity. We have chosen to represent this data in a custom C++ object named Test Object. The Test Object consists of one array of 5 integer values, 4 String objects from the C++ standard library, 3 integer values, and 3 float values. In total, this object contains 11 fields that are to be serialized and deserialized by the four libraries.

The Test Bench program starts with the user selecting the number of times they would like each library to serialize and then deserialize the randomly generated Test Object. The serialization process is as follows. We have developed a custom C++ class for each of the third-party libraries that are being tested. Each class has a serializeTestObject function and a deserializeTestObject function. These functions are implemented differently for each library, as they all have different procedures and syntax to serialize data. Inside of the Test Bench program, a call is made to serialize and deserialize the Test Object for as many times as the user selected upon running the program via the command line. When the Test Bench makes a serializeTestObject call to the custom C++ class developed for a library, that class will then write the data (with the type of serialization depending on the library) to a given file in a designated archives directory. When the Test Bench calls deserializeTestObject, it gives the function a file path and a reference to an empty Test Object. The custom class for the library will then read in the serialized data from the given file, and populate the empty Test Object with this data.

The Test Bench measures the time that these functions take for each library, and the output includes an average time for serialization and deserialization of the Test Object for each library. Output also includes the file size for the file containing the serialized data that each library creates. The final element of the output is a confirmation if the data has been properly serialized and deserialized (i.e. the original Test Object and the one that each library serialized and deserialized are the same). The Test Bench also includes a verbose option. With verbose mode enabled, the output will display all of the data for each deserialized Test Object from each library.

Otherwise, the output will simply include a statement saying that the deserialization was done correctly if it matches the original Test Object.



The structure of the development environment is an important element of the program. The directory structure can be found on our GitHub repository. On the top level, there are directories named archives, build, environmentSetup, inputData, libs, outputs, and resources. There is also a .gitignore file, CMakeLists.txt for cmake configuration, a README.md, and testBench.cpp, our main executable program. The purposes of each directory are as follows:

- archives - Serialized files (.bin, .xml, or .json) are stored here
- build - An empty directory for organization. Developers are encouraged to run make commands in this directory to keep the generated files organized. More information about building the executable can be found inside README.md
- environmentSetup - Contains a directory called bins_and_dirs where all of the libraries' include folders are located, as well as the necessary compiled binaries that they need to run
- inputData - Contains source files and headers for the Test Object class used in the Test Bench. Also includes the Clock object used in early development.

- libs - The directory containing the custom C++ classes implementing each library. Also contains the files generated by the flatc and protoc compilers from flatbuffers and protobuf, respectively.
- outputs - A directory containing the outputs of Test Bench runs for data analysis
- resources - A directory created by our project sponsor with helpful resources

The Test Bench was developed within a virtual machine running the Linux/Ubuntu Operating System. The specific virtual machine our team used was the Oracle VM VirtualBox and the version of Ubuntu that was used was 22.04.

Chosen Libraries & Previous Considerations

The only library that was looked into but not incorporated into the final project was Cap'n Proto. This was due to complications with the testing script given by the creators of Cap'n Proto having coding errors rendering it useless. Without being able to test installation, it was concluded that the product would be too problematic and the creators not reliable enough to continue to incorporate it for the rest of the project.

There were four libraries chosen for the project and incorporated in the test bench. Although raw serialization is very condensed and quick, it struggles with pointers since raw serialization copies bytes directly and cannot process pointer references correctly causing issues for the receiver of the data. The libraries are a good alternative since they have built in functions to allow pointers to be serialized.

Boost is a C++ exclusive library that prioritizes widespread choices that use templates to tackle as many problems as possible. The Test Bench utilizes Boost by having it serialize and deserialize with binary serialization. Boost was chosen to test how a library that emphasizes its widespread availability performs and what tradeoffs come at that expense. The Boost license makes it very easy to implement without worrying about restrictions and it has pre-built packages for many Linux machines. Boost contains many header files as well as a compiled binary file that the compiler must be linked to.

Cereal trades off availability for more efficient serializing. It is only applicable with C++ and has issues with object tracking, but trades off to be faster and minimal. The Test Bench utilizes Cereal to serialize in XML and JSON format along with Binary. Cereal was chosen because of the efficiency it advertises it has. Testing a library that cannot be used in all cases but is faster and takes up less space can be beneficial to see how useful the tradeoff is. Cereal is a header-only library, requiring nothing to be built by the user. As long as these headers are in the compiler's include path, Cereal will run.

Protobuf was designed as a replacement for XML and JSON that trades off readability to create a smaller and faster format of serialization. It does this by utilizing .proto files that make it possible to serialize across different projects and languages. In the Test Bench Protobuf uses a Binary format to serialize and deserialize input data. Protobuf was chosen since it is a library that does not focus primarily on C++ and instead puts its efforts into compatibility with outside code no matter their language. Testing Protobuf will show what issues arise from having files that are capable of being so compatible.

Flatbuffers is similar to Protobuf but made to be more efficient by eliminating the allocations that come with Protobuf. Although not as widely supported, Flatbuffers makes up for it by using less memory and having easier ways to access the data by only using buffers. In the Test Bench Flatbuffers utilized Binary format to serialize and deserialize the input data. Flatbuffers was chosen as a contrast to Protobuf to see how minimal differences in libraries changes how the two perform in memory, speed, etc. Testing Flatbuffers will also highlight how the differences in observed qualities contribute to the quality of a library.

Observational and Researched Qualities Analysis

Ease of Use:

When referring to the Ease of Use of a library, the main three points that come into play are:

- How easy it is to download a library
- How easy it is to implement a library

Downloading a library is the simplest step, which includes finding the right compressed file to download, but there can be several steps afterwards depending on how that file is formatted. Past that, implementing each library comes with the difficulty of finding the correct #include statements and lines of code to have the compiler know where the headers and compiled binaries are. Once a library is implemented into the program, it can still be difficult to work around it so that the code works correctly. This could be due to headers of functions having to be adjusted, new file types being introduced, etc.

Downloading Libraries:

Downloading Boost came with a few extra steps since it comes in a compressed file. They included every step in their Boost page "Getting Started on Unix Variants", which is 7 steps that goes from downloading to testing Boost, and every possible error that could come up in between.

Cereal's download is very straightforward and takes little to no time or difficulty. The code for it is pulled right off of GitHub and is made up of nothing but header files so there are no extra steps to get it up and running.

Protobuf's download was the most difficult of the bunch. The biggest issue that arose was the fact that there were many versions that were not compatible with the version of Ubuntu we worked on. This meant we had to go through and test several versions until we got one to work. Also, there was very little documentation on this error online. This was a time-consuming issue for our group.

Flatbuffers had clean and concise documentation making it easy to download. All of the header files were found on their Github and easy to copy over, and their tests were found right underneath on their website.

Implementing Libraries:

To implement Flatbuffers and ProtoBuf into the Test Bench a compiler had to be built which was tougher to find in the documentation than the header files. This was the same for building the schema around the file and serializing the data, as they both came from a learning curve from the documentation Flatbuffers provides. In

Flatbuffers, a schema file must be written and compiled through the flatc compiler in order to generate the headers and methods to serialize and deserialize. The same process must occur for ProtoBuf but with their protoc compiler.

The implementation of the other libraries (Boost and Cereal) went fairly smoothly utilizing CMake. As mentioned earlier in the section,all that was needed to do was:

1. Have the correct #include lines in the C++ files
2. Have the correct lines in the CMakeLists.txt files to have the compiler look for the headers and compiled binaries

Boost requires a .serialize() method to be written for each object that it is going to serialize. This is so Boost knows what fields of each object it is going to serialize. This is highly customizable and easy to write. Boost's documentation gives plenty of examples for how to write this method. There are also plenty of different ways to implement this functionality. You can split this functionality into a save() method and a load() method if desired. However, if it is implemented simply using the serialize() method, Boost can determine if it is storing or reading from the archive based on the context of the function call.

Implementing new inputs to work with Cereal is incredibly simple, it only needs the correct header files and proper #include statements to be able to serialize all the data types that are used in the input object. The learning curve comes from finding all of the header files in Cereal's include directory. Cereal can serialize primitive data types without additional #include statements. However, for more complex data types in the C++ standard library, such as strings, maps, arrays, queues, and several others, the program must include the appropriate header file from Cereal's "types" directory. Cereal also pairs well with Boost as it was programmed to be able to use Boost's serialization functions without having to alter the code, so if Boost works, then with a couple of #include statements, Cereal will work also.

Security Concerns:

When looking into security concerns for the versions of the libraries that were used in the Test Bench, none were found. Websites such as cvedetails and reports on GitHub were looked into during research and only past versions had reports on some security concerns. CVE Details uses two scoring systems, EPSS and CVSS, to evaluate certain concerns. According to FIRST.org, which is the Forum of Incident Response and Security Teams,

- EPSS, or Exploit Prediction Scoring System, is defined as: "a data-driven effort for estimating the likelihood (probability) that a software vulnerability will be exploited in the wild."
- CVSS, or Common Vulnerability Scoring System, is defined as: " a way to capture the principal characteristics of a vulnerability and produce a numerical score reflecting its severity. The numerical score can then be translated into a qualitative representation (such as low, medium, high, and critical) to help organizations properly assess and prioritize their vulnerability management processes."

  Page 7 contains the most notable concerns reported on CVE Details for all the libraries:

Library	Total Vulnerablities Reported	Highest CVSS and EPSS	Most Recent	Vulnerablities Reported for TestBench Version?
Boost Security	4	DATE: 1/17/2008 CVSS: 5.0 EPSS: 1.77% DETAILS: "The get_repeat_type function in basic_regex_creator.hpp in the Boost regex library (aka Boost.Regex) in Boost 1.33 and 1.34 allows context-dependent attackers to cause a denial of service (NULL dereference and crash) via an invalid regular expression"	DATE: 3/12/2013 CVSS: 5.0 EPSS: 0.76% DETAILS: " boost::locale::utf::utf_traits in the Boost.Locale library in Boost 1.48 through 1.52 does not properly detect certain invalid UTF-8 sequences, which might allow remote attackers to bypass input validation protection mechanisms via crafted trailing bytes."	VERSION: 1.82.0 NONE
Cereal Security	2	DATE: 3/30/2020 CVSS: 9.8 EPSS: 0.52% DETAILS: "An issue was discovered in USC iLab cereal through 1.3.0. It employs caching of std::shared_ptr values, using the raw pointer address as a unique identifier. This becomes problematic if an std::shared_ptr variable goes out of scope and is freed, and a new std::shared_ptr is allocated at the same address. Serialization fidelity thereby becomes dependent upon memory layout. In short, serialized std::shared_ptr variables cannot always be expected to serialize back into their original values. This can have any number of consequences, depending on the context within which this manifests."	SAME AS HIGHEST	VERSION: 1.3.2 NONE
Flatbuffers Security	2	DATE: 12/31/2020 CVSS: 9.8 EPSS: 0.22% DETAILS: "An issue was discovered in the flatbuffers crate before 0.6.1 for Rust. Arbitrary bytes can be reinterpreted as a bool, defeating soundness."	SAME AS HIGHEST	VERSION: 23.5.26 NONE
ProtoBuf Security	2	DATE: 9/25/2017 CVSS: 8.8 EPSS: 1.68% DETAILS: "protobuf allows remote authenticated attackers to cause a heap-based buffer overflow."	DATE: 1/26/2022 CVSS: 6.5 EPSS: 0.05% DETAILS: " Nullptr dereference when a null char is present in a proto symbol. The symbol is parsed incorrectly, leading to an unchecked call into the proto file's name during generation of the resulting error message. Since the symbol is incorrectly parsed, the file is nullptr. We recommend upgrading to version 3.15.0 or greater."	VERSION: 3.12 Potentially, 3.15 and above has been recommended

Community Support:

Each Library has at least 1 Discussion Group that the community and teams from the Libraries can collaborate on issues together. Boost provides the least collaborative platform with only letting the community subscribe to different mailing lists. Unlike the rest of the discussion groups you must be a subscriber to pose questions and receive answers. There is an archived list of conversations you can view without subscribing, however it is difficult to fluidly navigate through. For more common questions, they do provide a FAQ page that can direct you to specific resources.

Cereal provides a Google Groups and a StackOverflow Page. It seems to be actively used by the community and you're able to view past conversations and search for keywords, but upon review, it seems unreliable. Questions are rarely answered directly, and if so, they are either instantly answered or there's a large delay with no in between.

The same issue applies to ProtoBuf which also provides only a Google Groups and a Stack Overflow. The Google Groups are rarely answered with questions going unanswered for months. StackOverflow has questions more consistently answered and with wider collaboration.

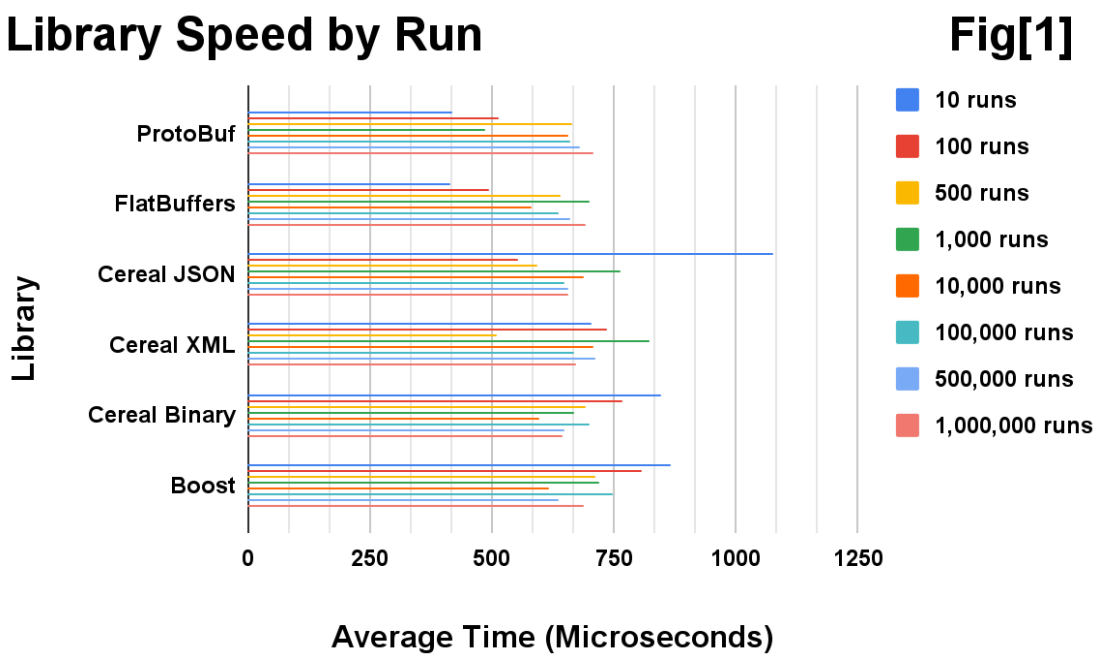
FlatBuffers supplies their community with the most resources such as a Google Groups and a StackOverFlow like the previous Libraries, along with a Discord Server and a Gitter Chat Room. They also have an Online Resources section on their Overview page with additional resources such as informational videos and additional Documentation to name a few. Like the other Libraries, their StackOverflow is only occasionally answered, but it seems more active than the previous two. Past 2023, their Google Groups are no longer taking in any new questions, however previous conversations are still able to be viewed. They are directing people to utilize StackOverflow or Discord instead. Their Discord is the most active resource available for FlatBuffers and most questions are answered in a timely manner.

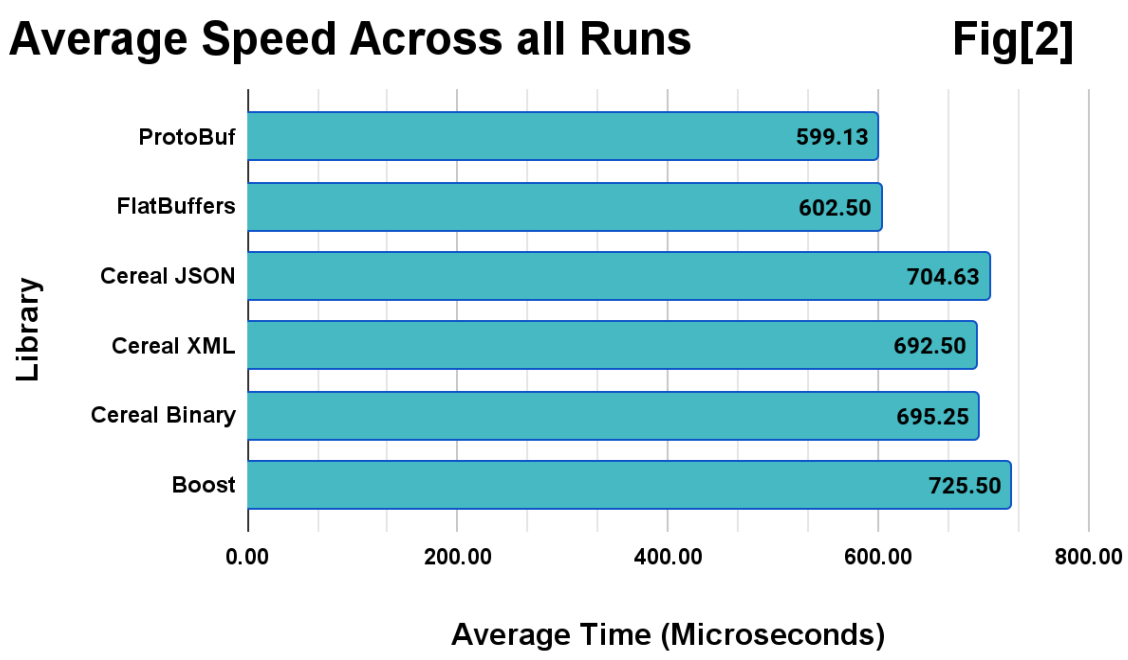
Metric Analysis Using the Test Bench

Performance

Speed

The generated data from testing the Libraries for their combined serialization and deserialization speeds within the Test Bench are visualized in Fig[1] and Fig[2] below.

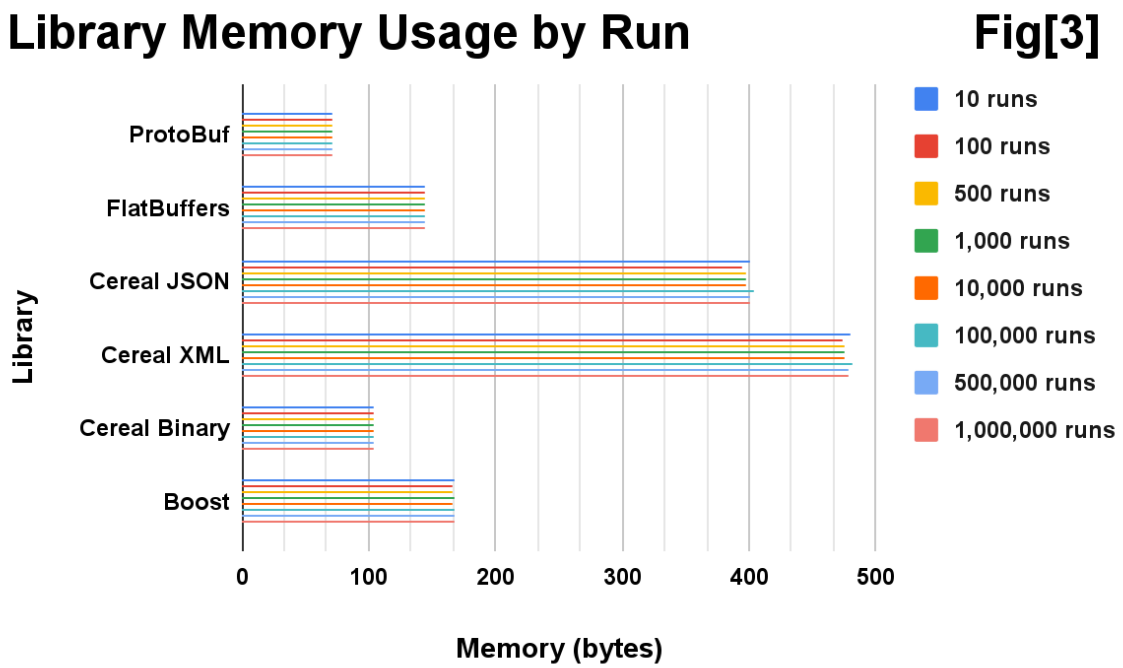


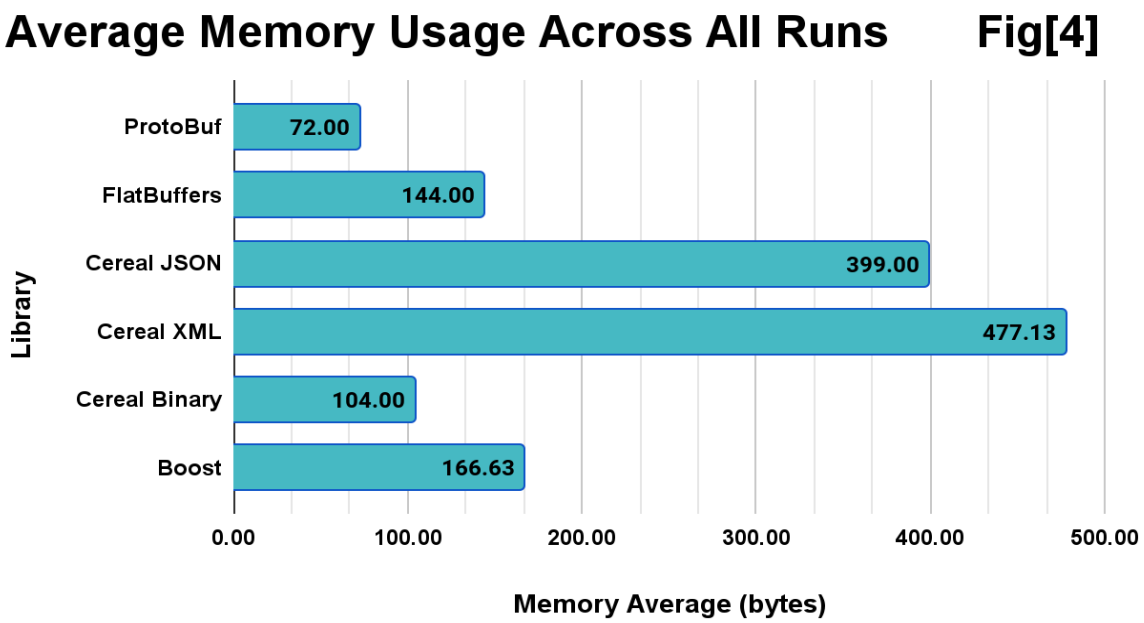


Based on this output we determined ProtoBuf, with an average time of 599.13 microseconds across all runs, to be the fastest library overall. This result aligns with what they promote as their advantages in their Overview: "It's like JSON, except it's smaller and faster,". Directly following ProtoBuf, with a difference of 3.37 microseconds, is FlatBuffers at 602.50 microseconds which also isn't unexpected as they also promote speed as one of their strengths. Following FlatBuffers are all the Cereal representations. Cereal XML at 692.50 microseconds, was 2.75 microseconds faster than Cereal Binary at 695.25 microseconds, This outcome was slightly unexpected as Binary was thought to be one of the faster serialization methods. Even though there's such a slight difference it was thought the Binary representation would still have the small lead. Cereal JSON was the slowest Cereal representation at 704.63 microseconds. However, these results did align with a statement on their site that promotes "In simple performance tests, cereal is usually faster than Boost's serialization library", as Boost did have the slowest average speed at 725.50 microseconds. Cereal's documentation notes that their Binary implementation captures less metadata than Boost's, making it smaller and faster. Though it's important to note that speed wasn't promoted as one of the Goals for Boost's system.

Memory Usage

The generated data from testing the Libraries for their Memory Usage within the Test Bench are visualized in Fig[3] and Fig[4] below.





It was revealed through the output that ProtoBuf used the least amount of memory at an impressively consistent 72 bytes. This shows another value that ProtoBuf promotes in their Overview such as, "Compact data storage". Cereal Binary follows next with another consistent 104 bytes and is the smallest out of the other Cereal Representations. FlatBuffers is the third at 144 bytes which isn't unsurprising as FlatBuffers also promotes memory efficiency, explaining in their Overview: "The only memory needed to access your data is that of the buffer. It requires 0 additional allocations.". There's a 22.63 byte difference between the memory averages of FlatBuffers and Boost with Boost using an average of 166.63 bytes. Which isn't that poor considering memory efficiency isn't one of their Goals in their Overview. Cereal JSON comes next using an average of 399.00 bytes and following right after is the largest memory user, Cereal XML, at an average of 477.13 bytes. The results on Cereal's representations aren't that unexpected, the few comments on their JSON and XML parsers in their Overview was that they were some of the fastest available in C++. While it was only their Binary representation that they labeled as taking up less space than Boost and that lined up with our Test Bench results as well.

Reliability/Data Integrity

To test Data Integrity each pre-serialized and post-serialized file were tested to see if their contents matched after the serialization process. This was tested with varying quantities of Runs as shown in the Table below.

Is the Object Correctly Deserialized?								
Library	10 runs	100 runs	500 runs	1,000 runs	10,000 runs	100,000 runs	500,000 runs	1,000,000 runs
ProtoBuf	yes	yes	yes	yes	yes	yes	yes	yes
FlatBuffers	yes	yes	yes	yes	yes	yes	yes	yes
Cereal JSON	yes	yes	yes	yes	yes	yes	yes	yes
Cereal XML	yes	yes	yes	yes	yes	yes	yes	yes
Cereal Binary	yes	yes	yes	yes	yes	yes	yes	yes
Boost	yes	yes	yes	yes	yes	yes	yes	yes

What was found was that no matter the amount of runs or the differences between the libraries, each library was equally reliable and able to maintain the contents of the object it was serializing.

Overall Library Recommendation

Through the process of testing each of the serialization Libraries and implementing each one into the Test Bench, two Libraries stood out from the rest when observing all of the different characteristics.

The first is ProtoBuf, which stands out when it comes to both of the numerical factors. The average speed is the fastest by a small margin, about 600 microseconds, which is 225 microseconds faster than the slowest of the libraries tested, Boost. In regards to memory, ProtoBuf creates the most condensed files by a large margin. On average, it uses 72 bytes which is over 400 bytes less than the worst library in this category, Cereal XML, and 30 bytes from the next most condensed, Cereal Binary. The main drawback is that ProtoBuf was found to be the most difficult to use in our project and has limited resources for community help.

FlatBuffers is also highly recommended because of the limited drawbacks that it has. FlatBuffers is practically the fastest, being only 2 and half microseconds on average behind ProtoBuf, and FlatBuffers comes in 3rd out of 6 for memory usage, although that does come with double the amount of storage as ProtoBuf. FlatBuffers shine in the nonnumerical factors observed in the project. The Library caused us very few issues when implementing, has good documentation, and has the best and most community resources making it the best to find a solution if any problems do arise.

If a situation arises where speed and memory are not a concern, then Cereal is most likely going to give developers the easiest and smoothest experience. With nothing to build, since it is header-only, it is extremely easy to integrate. This is also due to Cereal's fantastic documentation and walkthroughs.

Works Cited

"Before You Get Started." FlatBuffers, flatbuffers.dev/flatbuffers_guide_use_cpp.html.

Boost C++ Libraries, www.boost.org/community/groups.html.

"Boost FAQ." Boost C++ Libraries, www.boost.org/users/faq.html.

"Cereal - A C++11 Library for Serialization." Cereal Docs - Main, uscilab.github.io/cereal/.

"Cereal Serialization Library." Google Groups, Google, groups.google.com/g/cerealcpp.

"CVE Security Vulnerability Database. Security Vulnerabilities, Exploits, References and More." CVE Security Vulnerability Database. Security Vulnerabilities, Exploits, References and More, www.cvedetails.com/.

"Flatbuffers." Google Groups, Google, groups.google.com/g/flatbuffers.

"FlatBuffers." Stack Overflow, stackoverflow.com/search?q=flatbuffers&s=75e81adb-f930-4260-9179-3affd12d01a4.

"Getting Started on Unix Variants." Boost C++ Libraries, www.boost.org/doc/libs/1_84_0/more/getting_started/unix-variants.html.

Google. "Flatbuffers/Include/Flatbuffers at Master · Google/Flatbuffers." GitHub, github.com/google/flatbuffers/tree/master/include/flatbuffers.

"Improving Security Together." FIRST, www.first.org/.

"Measure Execution Time of a Function in C++." GeeksforGeeks, GeeksforGeeks, 3 May 2023, www.geeksforgeeks.org/measure-execution-time-function-cpp/.

"Newest 'cereal' Questions." Stack Overflow, stackoverflow.com/questions/tagged/cereal.

"Newest 'protocol-Buffers' Questions." Stack Overflow, stackoverflow.com/questions/tagged/protocol-buffers.

"Overview." FlatBuffers, flatbuffers.dev/.

"Overview." Protocol Buffers Documentation, protobuf.dev/overview/.

"Protocol Buffers." Google Groups, Google, groups.google.com/g/protobuf. Serialization Overview, www.boost.org/doc/libs/1_84_0/libs/serialization/doc/index.html.